Listas

Mario A. Morales Rivera
mamorales@correo.unicordoba.edu.co
Universidad de Córdoba



Tabla de contenidos I

1. Listas
2. Indexado de listas
3. Ejemplo de Aplicación
4. Listas anidadas
5. Iteración en listas



Listas



¿Qué aprenderemos?

Estudiaremos como crear listas con la función `list`, como acceder a los elementos de una lista veremos un ejemplo de aplicación.



Definición

Una lista es, en términos generales, es un vector donde cada elemento puede ser de un tipo diferente.

Esta clase trata sobre cómo crear, indexar y manipular listas.



Creación de listas

Crearemos una lista usando la función `list`

```
l<-list(
c(1, 1, 2, 5, 14, 42),
month.abb,
matrix(c(3, -8, 1, -3), nrow = 2),
asin
)
print(l)
```

```
[[1]]
[1]  1  1  2  5 14 42

[[2]]
 [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec

[[3]]
     [,1] [,2]
```



Nota

Nota: La función `asin` es la inversa de la función `sin` (seno), dado el seno de un angulo la función `asin` regresa en angulo, veamos

```
sin(pi/4)
```

```
[1] 0.7071068
```

```
asin(0.7071068)
```

```
[1] 0.7853982
```

```
# observe que es lo mismo que pi/4
pi/4
```

```
[1] 0.7853982
```



Nombres de elementos

Después de creada la lista se le pueden asignar nombres a sus elementos con la función `names`, de la siguiente forma

```
names(l)=c("vec1","meses","mat1","fun")
print(l)
```

```
$vec1
[1]  1  1  2  5 14 42

$meses
 [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "De

$mat1
     [,1] [,2]
[1,]    3    1
[2,]   -8   -3

$fun
```



Nombres de elementos

También podemos asignar los nombres de los elementos de la lista al momento de crearla, de la siguiente forma

```
li<-list(
vec1=c(1, 1, 2, 5, 14, 42),
meses=month.abb,
mat1=matrix(c(3, -8, 1, -3), nrow = 2),
fun=asin
)
print(li)
```

```
$vec1
[1]  1  1  2  5 14 42

$meses
 [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "De

$mat1
```



Agregaer elemento

```
df=data.frame(x=rbinom(6,10,0.3),g=c(1,2))
l[['data']]=df
```

- Equivalente a

```
df=data.frame(x=rbinom(6,10,0.3),g=c(1,2))
l$data=df
```



Indexado de listas



Formas de indexado

Como en el caso de los vectores, matrices, y `data frames` las listas se pueden indexar de varias formas

- Enteros positivos
- Caracteres
- Valores lógicos
- Enteros negativos



Indexado con indice numérico

Recuperamos el segundo elemento de la lista `l`

```
l[2]
```

```
$meses
 [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "De
```

Recuperamos los elementos segundo y tercero de la lista `l`

```
l[c(2,3)]
```

```
$meses
 [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "De


$mat1
     [,1] [,2]
[1,]    3    1
[2,]   -8   -3
```



Ejercicio

Se da la lista `l` cuyo primer elemento es un vector numérico, hallar la media de los valores en el vector.

```
l[1]
```

```
$vec1
[1]  1  1  2  5 14 42
```

```
mean(l[1])
```

```
[1] NA
```

```
class(l[1])
```

```
[1] "list"
```



La razón del NA

La función mean regresa un NA debido a que la forma de recuperar mediante l[1] regresa una lista y no un vector numérico como lo exige mean.

```
l[[1]]
```

```
[1]  1  1  2  5 14 42
```

```
mean(l[[1]])
```

```
[1] 10.83333
```

```
class(l[[1]])
```

```
[1] "numeric"
```



Indexado con un vector de nombres

Cuando la lista tiene asignados nombres a sus elementos, se puede acceder a ellos mediante

```
l[c('vec1')]
```

```
$vec1
[1]  1  1  2  5 14 42
```

```
class(l[c('vec1')] )
```

```
[1] "list"
```

note que en este caso también recuperamos una lista con el vector numérico por dentro.

```
l[c('vec1','fun')]
```

```
$vec1
[1]  1  1  2  5 14 42
```



Uso de [[]]

Para recuperar directamente el elemento use los nombres de la siguiente forma

```
l[['vec1']]
```

```
[1]  1  1  2  5 14 42
```



Uso de $

Otra forma de recuperar los elementos de la lista por los nombre es mediante el signo $ de la siguiente

```
l$mat1
```

```
     [,1] [,2]
[1,]    3    1
[2,]   -8   -3
```



Indexado con numeros negativos

Se trae todos los elementos excepto el que corresponde a la posición del negativo.

```
l[-c(2,4)] # trae todos excepto es 2 y el 4
```

```
$vec1
[1]  1  1  2  5 14 42

$mat1
     [,1] [,2]
[1,]    3    1
[2,]   -8   -3
```



Ejemplo

```
l[-1] # todos menos el primero
```

```
$meses
 [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"

$mat1
     [,1] [,2]
[1,]    3    1
[2,]   -8   -3

$fun
function (x)  .Primitive("asin")
```



Ejemplo

Recuperar el vector 1 y calcular su media.

```
l$vec1
```

```
[1]  1  1  2  5 14 42
```

```
mean(l$vec1)
```

```
[1] 10.83333
```



Indexado con un vector lógico

Si indexamos la lista con un vector lógico, obtenemos aquellos elementos de la lista que corresponden a TRUE como se muestra a continuación

```
# recuperamos el elemento 1 y 3
l[c(TRUE,FALSE,TRUE,FALSE)]
```

```
$vec1
[1]  1  1  2  5 14 42

$mat1
     [,1] [,2]
[1,]    3    1
[2,]   -8   -3
```



Ejemplo de Aplicación



Aplicación

Consideremos los siguientes datos

105	97	245	163	207	134	218	199	160	196
221	154	228	131	180	178	157	151	175	201
183	153	174	154	190	76	101	142	149	200
186	174	199	115	193	167	171	163	87	176
121	120	181	160	194	184	165	145	160	150
181	168	158	208	133	135	172	171	237	170
180	167	176	158	156	229	158	148	150	118
143	141	110	133	123	146	169	158	135	149

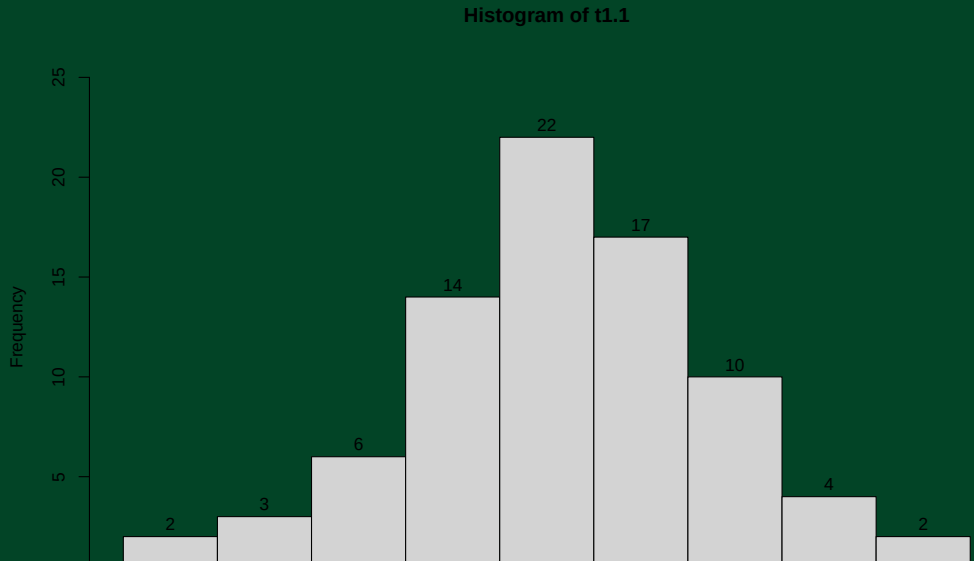
Ejercicio

- Elabore el histograma de los datos
- Imprima la tabla de frecuencias con que se elaboró el histograma
- Calcule la media a partir de los datos agrupados en la tabla anterior.



El histograma

```
hist(t1.1,breaks=seq(70,250,by=20),right = FALSE,labels=TRUE,ylim=c(0,25))
```



Solución

```
histo=hist(t1.1,breaks=seq(70,250,by=20),right = FALSE,labels=TRUE,ylim=c(
```

Es histo una lista ?

```
mode(histo)
```

```
[1] "list"
```

```
is.list(histo)
```

```
[1] TRUE
```



Contenido de la lista

Qué cosas tengo dentro de esa lista?

```
names(histo)
```

```
[1] "breaks"   "counts"   "density"  "mids"     "xname"    "equidist"
```



Los límites de los intervalos

```
histo$breaks
```

```
 [1]  70  90 110 130 150 170 190 210 230 250
```

```
Li=histo$breaks[-length(histo$breaks)]
Li
```

```
[1]  70  90 110 130 150 170 190 210 230
```

```
Ls=histo$breaks[-1]
Ls
```

```
[1]  90 110 130 150 170 190 210 230 250
```



Limites de los intervalos

```
cbind(Li,Ls)
```

```
      Li  Ls
 [1,]  70  90
 [2,]  90 110
 [3,] 110 130
 [4,] 130 150
 [5,] 150 170
 [6,] 170 190
 [7,] 190 210
 [8,] 210 230
 [9,] 230 250
```



Las marcas de clase

```
MC=histo$mids
cbind(Li,Ls,MC)
```

```
      Li  Ls  MC
 [1,]  70  90  80
 [2,]  90 110 100
 [3,] 110 130 120
 [4,] 130 150 140
 [5,] 150 170 160
 [6,] 170 190 180
 [7,] 190 210 200
 [8,] 210 230 220
 [9,] 230 250 240
```



Las frecuencias absolutas y relativas

```
fi=histo$counts
n=sum(fi)
fir=fi/n
cbind(Li,Ls,MC,fi,fir)
```

```
      Li  Ls  MC fi    fir
 [1,]  70  90  80  2 0.0250
 [2,]  90 110 100  3 0.0375
 [3,] 110 130 120  6 0.0750
 [4,] 130 150 140 14 0.1750
 [5,] 150 170 160 22 0.2750
 [6,] 170 190 180 17 0.2125
 [7,] 190 210 200 10 0.1250
 [8,] 210 230 220  4 0.0500
 [9,] 230 250 240  2 0.0250
```



Las frecuencias acumuladas y acumuladas relativas

```
Fi=cumsum(fi)
Fir=Fi/n
cbind(Li,Ls,MC,fi,fir,Fi,Fir)
```

```
      Li  Ls  MC fi    fir Fi    Fir
 [1,]  70  90  80  2 0.0250  2 0.0250
 [2,]  90 110 100  3 0.0375  5 0.0625
 [3,] 110 130 120  6 0.0750 11 0.1375
 [4,] 130 150 140 14 0.1750 25 0.3125
 [5,] 150 170 160 22 0.2750 47 0.5875
 [6,] 170 190 180 17 0.2125 64 0.8000
 [7,] 190 210 200 10 0.1250 74 0.9250
 [8,] 210 230 220  4 0.0500 78 0.9750
 [9,] 230 250 240  2 0.0250 80 1.0000
```



La tabla en formato latex

Li	Ls	MC	fi	fir	Fi	Fir
70	90	80	2	0.0250	2	0.0250
90	110	100	3	0.0375	5	0.0625
110	130	120	6	0.0750	11	0.1375
130	150	140	14	0.1750	25	0.3125
150	170	160	22	0.2750	47	0.5875
170	190	180	17	0.2125	64	0.8000
190	210	200	10	0.1250	74	0.9250
210	230	220	4	0.0500	78	0.9750
230	250	240	2	0.0250	80	1.0000



Aplicación

La media calculada a partir de datos agrupados en un histograma se calcula con la siguiente formula

$$\bar{X}_a = \frac{\sum_{i=1}^{k} M_i \times f_i}{n}$$

Donde M_i es la marca de la clase i, f_i es la frecuencia, n es el numero de datos y k es el numero de intervalos. Recordemos que $n = \sum f_i$.



Implementación

A continuación se implementa la formula en R

```
sum( histo$counts*histo$mids )/sum(histo$counts)
```

```
[1] 163.5
```



La media de los datos sin agrupar es 162.6625, se observa que no hay mucha diferencia entre las dos medias, por lo tanto la media calculada a partir de los datos agrupados en el histograma es una buena aproximación de la media real de los datos.



Ejercicio

Calcular la varianza a partir de los datos agrupados y compararla con la varianza de los datos sin agrupar.

Nota: La varianza con datos agrupados se obtiene con la siguiente formula

$$S_a^2 = \frac{\sum (M_i - \bar{X}_a)^2 \times f_i}{n-1} = \frac{\sum M_i^2 f_i - n\bar{X}_a^2}{n-1}$$



Listas anidadas



Listas Anidadas

- Dado que una lista puede contener **cualquier** tipo de objeto en R, ¡también puede albergar otras listas en su interior!
- A esto se le conoce como una lista anidada.
- Esta característica es muy poderosa porque nos permite crear estructuras jerárquicas complejas.



Ejemplo: lista anidada

```r
# Creamos una lista que contiene otra lista internamente
curso <- list(
  asignatura = "Estadística Computacional",
  semestre = "2026-2",
  notas = list(
    corte1 = c(4.5, 3.8, 4.0),
    corte2 = c(3.2, 4.1, 4.8)
  )
)
```



Indexado en Listas Anidadas

Para acceder a los elementos profundos (“sub-elementos”) de una lista anidada, simplemente debemos encadenar los operadores de extracción, ya sea el signo $ o los dobles corchetes `[[]]`.

```
# Encadenando el signo $ (Muy intuitivo)
curso$notas$corte1
```

```
[1] 4.5 3.8 4.0
```



Con dobles corchetes [[]]

```
# Encadenando dobles corchetes (Elemento 3, sub-elemento 2)
curso[[3]][[2]]
```

```
[1] 3.2 4.1 4.8
```



Ejercicio

Calcular la nota promedio del segundo corte

```
# Calculando la media de un sub-elemento
mean(curso$notas$corte2)
```

```
[1] 4.033333
```



Iteración en listas



Iteración en Listas: Familia `apply`

En R, es muy común querer realizar la **misma operación matemática o estadística** sobre cada uno de los elementos de una lista.

Aunque esto se puede hacer con un bucle `for`, en R la mejor práctica (y la más rápida) es usar la familia de funciones `apply`. Hoy nos enfocaremos en dos de las más importantes para listas:

- `lapply()`: Devuelve siempre una **l**ista.
- `sapply()`: Devuelve un resultado **s**implificado (usualmente un vector).



La función `lapply()`

La función `lapply()` (List Apply) toma una lista, aplica una función a cada uno de sus elementos y **siempre devuelve otra lista** de la misma longitud.

```
# Creamos una lista con tres vectores numéricos
notas_estudiantes <- list(
  Juan = c(4.0, 3.5, 4.2),
  Maria = c(4.8, 5.0, 4.6),
  Pedro = c(3.1, 2.8, 3.5)
)

# Calculamos la media de cada estudiante
medias_lista <- lapply(notas_estudiantes, mean)
medias_lista
```

```
$Juan
[1] 3.9

$Maria
```



La función `sapply()`

- La función `sapply()` (Simplified Apply) hace exactamente el mismo cálculo que `lapply()`, pero intenta simplificar el resultado final.
- Si el resultado de la función es un solo número por cada elemento (como la media), `sapply()` lo agrupa todo elegantemente en un vector numérico.



Uso de sapply()

```r
# Mismo cálculo, pero con sapply
medias_vector <- sapply(notas_estudiantes, mean)
medias_vector
```

```
    Juan    Maria    Pedro
3.900000 4.800000 3.133333
```



Argumentos adicionales en apply

- Tanto `lapply()` como `sapply()` permiten pasarle argumentos adicionales a la función que estamos aplicando.
- Por ejemplo, si uno de los estudiantes faltó a un examen y tiene un NA:

```
notas_con_na <- list(Juan = c(4.0, 3.5, 4.2), Ana = c(4.5, NA, 4.0))

# Agregamos na.rm = TRUE como tercer argumento
sapply(notas_con_na, mean, na.rm = TRUE)
```

```
Juan  Ana
3.90 4.25
```

